

Cleanup API

1. **uploadController.js**: Receives an iPhone HEIC or Android JPG, uses **Sharp** to make it a standard 1200px JPEG, and saves it to the **Job** model.
2. **archiveController.js**: Receives the Job ID and triggers the service layer.
3. **jobService.js**: Uses **Puppeteer** to "print" those standard JPEGs into a luxury monochrome PDF, saves the PDF to Firebase, and creates a **Report** document using the schema above.
4. **storageService.js**: Deletes the original JPEGs to keep your costs low while keeping the PDF safe.

Why this is "Production-Ready"

- **Data Integrity**: Because we use a **Mongoose Session**, if the PDF fails to upload, the job status doesn't change and photos aren't deleted.
- **Searchable History**: By saving the **clientName** in the Report model, you can quickly show a "Client Portal" later where they can download all their old cleanup certificates.
- **Optimized Performance**: Sharp handles the mobile image "mess," and Puppeteer handles the high-end output.

This Blueprint outlines the complete **Image Normalization and Atomic Cleanup Architecture**. This design ensures that **RoadResQ** remains a high-performance system by handling the "iPhone Problem" early and protecting data integrity with database transactions.

1. The Normalization Strategy (The "Ingress")

Instead of dealing with varying formats like HEIC or massive raw files at the time of report generation, we normalize everything during the **Upload Phase**.

- **Format Conversion**: The system detects Apple's **.heic** format and converts it to a standard **.jpg** using **heic-convert**.
- **Dimensional Re-shaping**: Every image is resized to a maximum of **1200px** on its longest side via **sharp**. This keeps the aspect ratio intact while ensuring the final PDF isn't a massive, un-sendable file.
- **Compression**: Images are saved at **80% quality** to balance visual clarity with storage efficiency.

2. The Storage Layer (The "Vault")

Your `storageService.js` acts as a specialized gateway for Firebase.

- **Buffer Processing:** It accepts raw data "Buffers" directly from memory. This means images and PDFs never touch your server's local disk, making it faster and more secure.
- **Path Logic:** It organizes files into folders (`jobs/` for active work and `reports/` for permanent records).
- **URL Parsing:** It contains a specialized "Delete" logic that can take a public Firebase URL and extract the internal path to purge the file safely.

3. The Atomic Transaction (The "Bridge")

The **Archive Service** is the most critical part of the blueprint. It uses a **Mongoose Session** to ensure that "nothing is lost, or nothing is changed".

1. **Start Transaction:** The database creates a "safety net".
2. **Generate PDF:** Puppeteer creates a luxury monochrome PDF using the standardized JPEGs.
3. **Create Report:** A permanent `Report` document is created in the database with a link to the PDF.
4. **Archive Job:** The `Job` status is set to `archived` and its `photos` array is emptied.
5. **Commit:** If all the above steps succeed, the changes are finalized. If any step fails (e.g., Firebase is down), the transaction **rolls back** everything so no photos are deleted by mistake.

4. The Purge Logic (The "Cleanup")

Only **after** the database transaction is successfully committed do we trigger the physical deletion of the original photos from Firebase.

- **Cost Efficiency:** By deleting the individual JPEGs and keeping only the single PDF, you save on long-term storage costs.
- **Performance:** The cleanup happens "asynchronously" so the user doesn't have to wait for the cloud to delete 20 photos before seeing their "Success" message.

5. The Permanent Record (The "Egress")

The final output is a document in your **Reports Collection**.

- **Frozen State:** Unlike a "live" job, a Report is a snapshot of history.
- **Client Access:** It stores the `clientName` and `reportUrl` so you can easily build a "Past Reports" dashboard for your Qatari clients in the future.

This blueprint transforms a simple "delete" function into a professional, automated logistics workflow.